

```

import '@splidejs/splide/dist/css/splide-core.min.css';
import '../scss/style.scss';
import Splide from '@splidejs/splide';
import onChange from 'on-change'; // Установите через: npm install on-change

// [Бизнес-логика]: Изолированная функция проверки данных (чистая функция)
const validateForm = (values) => {
  const errors = {};
  if (!values.name.trim()) {
    errors.name = 'field cannot be empty';
  }
  if (!/^\+7\d{10}$/.test(values.phone)) {
    errors.phone = 'the correct phone format +79999999999';
  }
  if (!/^\S+@\S+\.\S+$/.test(values.email)) {
    errors.email = 'incorrect email format';
  }
  return errors;
};

// [Инфраструктура]: Оптимизация отображения элементов при скролле
const observer = new IntersectionObserver(
  (entries) => {
    entries.forEach((entry) => {
      entry.target.classList.toggle('hide', !entry.isIntersecting);
      if (entry.isIntersecting) observer.unobserve(entry.target);
    });
  },
  { threshold: 0.7 }
);

const configSplideSpeakers = {
  type: 'slide',
  perPage: 3,
  autoplay: false,
  pagination: false,
  arrows: false,
  gap: '20px',
  breakpoints: {
    1200: { perPage: 2, pagination: false, arrows: true, gap: '20px' },
    640: { perPage: 1, gap: '0px' }
  }
};

function initSplide() {
  const sliderSpeakers = new Splide('#slider-speakers', configSplideSpeakers);
  sliderSpeakers.mount();
}

function initAnimation() {
  const targets = document.querySelectorAll('.js-animate');
  targets.forEach((target) => {
    target.classList.remove('js-animate');
    target.classList.add('hide');
    setTimeout(() => { observer.observe(target); }, 500);
  });
}

function handleDOMContentLoaded() {
  const form = document.getElementById('appointment-form');
  const submitBtn = form.querySelector('button[type="submit"]') || form.querySelector('input[type="submit"]');

  // 1. [Слой MODEL]: Состояние полей, ошибок и фазы процесса формы
  const rawState = {
    values: { name: '', phone: '', email: '' },
    errors: {},
    status: 'filling' // Возможные статусы: 'filling', 'sending', 'success'
  };

  // 3. [Слой VIEW]: Декларативная синхронизация изменений Модели с DOM-деревом
  const render = (path, value) => {
    // Реагируем на изменение статуса отправки (конечный автомат)
  }
}

```

```

if (path === 'status') {
  if (value === 'sending') {
    if (submitBtn) submitBtn.disabled = true;
  }
  if (value === 'filling') {
    if (submitBtn) submitBtn.disabled = false;
  }
  if (value === 'success') {
    document.getElementById('success-message').innerText = 'The form has been successfully submitted!';
    form.reset();
    if (submitBtn) submitBtn.disabled = false;
    setTimeout(() => { document.getElementById('success-message').innerText = ''; }, 1000);
  }
}

// Реагируем на точечное изменение/сброс объекта ошибок
if (path.startsWith('errors')) {
  const fieldNames = ['name', 'phone', 'email'];
  fieldNames.forEach((name) => {
    const input = document.getElementById(name);
    const errorElement = document.getElementById(`${name}-error`);
    const currentError = state.errors[name];

    if (currentError) {
      input?.classList.add('is-invalid');
      if (errorElement) errorElement.innerText = currentError;
    } else {
      input?.classList.remove('is-invalid');
      if (errorElement) errorElement.innerText = '';
    }
  });
}

// Связываем Модель и Представление через прокси-библиотеку
const state = onChange(rawState, (path, value) => render(path, value));

// 2. [Слой CONTROLLER 1]: Валидация "на лету" при вводе в инпуты
form.addEventListener('input', (event) => {
  const { id, value } = event.target;
  if (id in state.values) {
    state.values[id] = value;
    state.errors = validateForm(state.values); // При изменении свойства errors сработает render()
  }
});

// 2. [Слой CONTROLLER 2]: Управление асинхронной отправкой формы на сервер
form.addEventListener('submit', async (event) => {
  event.preventDefault();

  // Валидация непосредственно перед отправкой
  const currentErrors = validateForm(state.values);
  state.errors = currentErrors;

  if (Object.keys(currentErrors).length > 0) return;

  state.status = 'sending'; // Автоматически заблокирует кнопку во View
  try {
    // Имитация fetch/axios сетевого запроса на бэкенд
    // await fetch('/api/appointment', { method: 'POST', body: JSON.stringify(state.values) });
    await new Promise((resolve) => setTimeout(resolve, 1000)); // Задержка сети в 1 секунду
    state.status = 'success'; // Переключит экраны успеха во View
  } catch (error) {
    state.status = 'filling'; // Разблокирует кнопку в случае сбоя сети
  }
});

// [Инфраструктура интерфейса]: Бургер-меню, поведение скrolла (оставлено в исходном виде)
const burger = document.getElementById('burger');
const headerButton = document.getElementById('header-button');
const nav = document.getElementById('desktop-nav');
const headerLink = nav.querySelectorAll('a');
const { body } = document;

```

```
const scrollBtn = document.getElementById('scroll-top-btn');

burger.addEventListener('click', () => {
  headerButton.classList.toggle('active');
  nav.classList.toggle('active');
  burger.classList.toggle('active');
  body.style.overflow = nav.classList.contains('active') ? 'hidden' : 'auto';
});

window.addEventListener('scroll', () => {
  scrollBtn.style.display = window.scrollY > 300 ? 'flex' : 'none';
});

scrollBtn.addEventListener('click', () => {
  window.scrollTo({ top: 0, behavior: 'smooth' });
});

initSplide();
initAnimation();
}

window.addEventListener('DOMContentLoaded', () => handleDOMContentLoaded(), { once: true });
```